

Rapport individuel — Itération 2

Mati Afriat (`mat6784`) — Projet Long SHDL

11 mai 2026

Contexte

Responsable du **simulateur** (axe *simulation*, branche `simulation`). Objectif sprint 2 : faire converger les deux modélisations explorées au sprint 1 vers une *interface unique* consommable par le parser d'Alexis, et étendre le langage supporté (opérations logiques, vecteurs de signaux, appel de module). Construction d'une nouvelle abstraction **Erwan** servant de pivot entre interprétation et simulation.

Activités réalisées

- **Package `erwan/`** (interface interprétation/simulation) : classe **Erwan** modélisant un signal comme une opération arborescente (un **Erwan** contient les autres **Erwan** dont il dépend + l'**Operation**), interface **Branchement**, **AppelModule** pour la composition, **Operation** {ET, OU, NON, XOR, IDENTITE, CONSTANTE}, **Descripteur** pour les vecteurs (range `indiceDebut..indiceFin`).
- **Conteneur Module** : agrège **Plan**: `List<Erwan>`, **Entrees/Sorties**: `List<Descripteur>`, **Branchements**: `List<AppelModule>`. Contrat consommé par `Conversion.convert` côté parser.
- **Support des vecteurs** (commit `vecteur fonctionnels`, 8 mai) et renforcement du simulateur historique : `Constante`, `ConnecteurListener`, classes `Experience1..6` pour validation expérimentale.
- **Documentation JavaDoc** générée (`erwan/*.html`, `simulateur/*.html`).

Résultats

- **Interface `erwan.Module` stabilisée** : utilisée comme cible par la conversion d'Alexis, tests bout-en-bout verts sur ET/OU/NOT/XOR.
- **Vecteurs fonctionnels** (commit `3993b11`) : ouvre la voie au support des bus dans la suite du projet.
- Un **bug `Erwan.OR()` retournant `Op.AND`** a été détecté côté parser via un test e2e XOR ; correction intégrée au sprint avec test sentinelle.

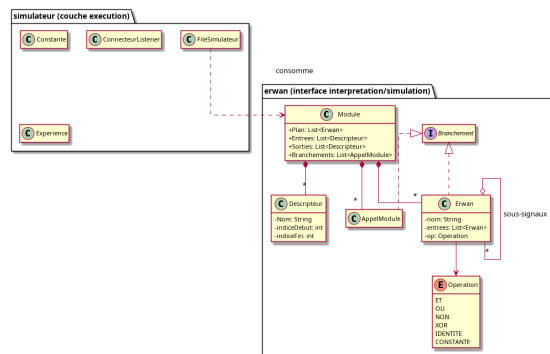


Figure — Diagramme de classes du package `erwan` (source `mati-sprint2-classes.puml`).

Difficultés et réussites

Difficulté : **réconcilier les deux modèles** du sprint 1 (Lien simple mais sans vecteurs/composition ; Connecteur expressif mais lourd). La classe **Erwan** a été construite comme *descripteur déclaratif* (qui dépend de quoi, via quelle opération) indépendant de l'exécution. Réussite : ce pivot rend l'intégration avec le parser triviale et a permis la livraison de *parse* → *convert* → *simulate* bout-en-bout.

Manuel utilisateur

Pré-requis : JDK 25, JUnit 4 et Hamcrest. Création d'un **Module** : instancier les **Erwan** via les méthodes de classe (pas de constructeur public), préciser **Entrees/Sorties** via **Descripteur**, passer le tout à `new FileSimulateur(module.Plan)`. Tests : `java org.junit.runner.JUnitCore erwan.TestErwan`. JavaDoc consultable dans `erwan/*.html` et `simulateur/*.html`. Code sur la branche `simulation`.